KOENIG

step forward



Microservice Architecture

Design principles, boundaries and responsibilities

What You Will Learn

1. What is a monolith
2. What are microservices, and why
3. Microservice design principles
4. Where do we cut? Introducing DDD
5. Language, subdomains, bounded contexts and Context mapping
6. Inside a service: entities, events, services

Demo Scenario: Learnly Online Learning

Learnly is an online learning platform. Students browse courses, enroll, pay, watch lessons, track progress and receive a certificate. Today everything is one application with one database. We design the services on paper before anyone writes code.

Monolith → **Principles** → **Boundaries** → **Services**

12 features, one database

what makes a good service

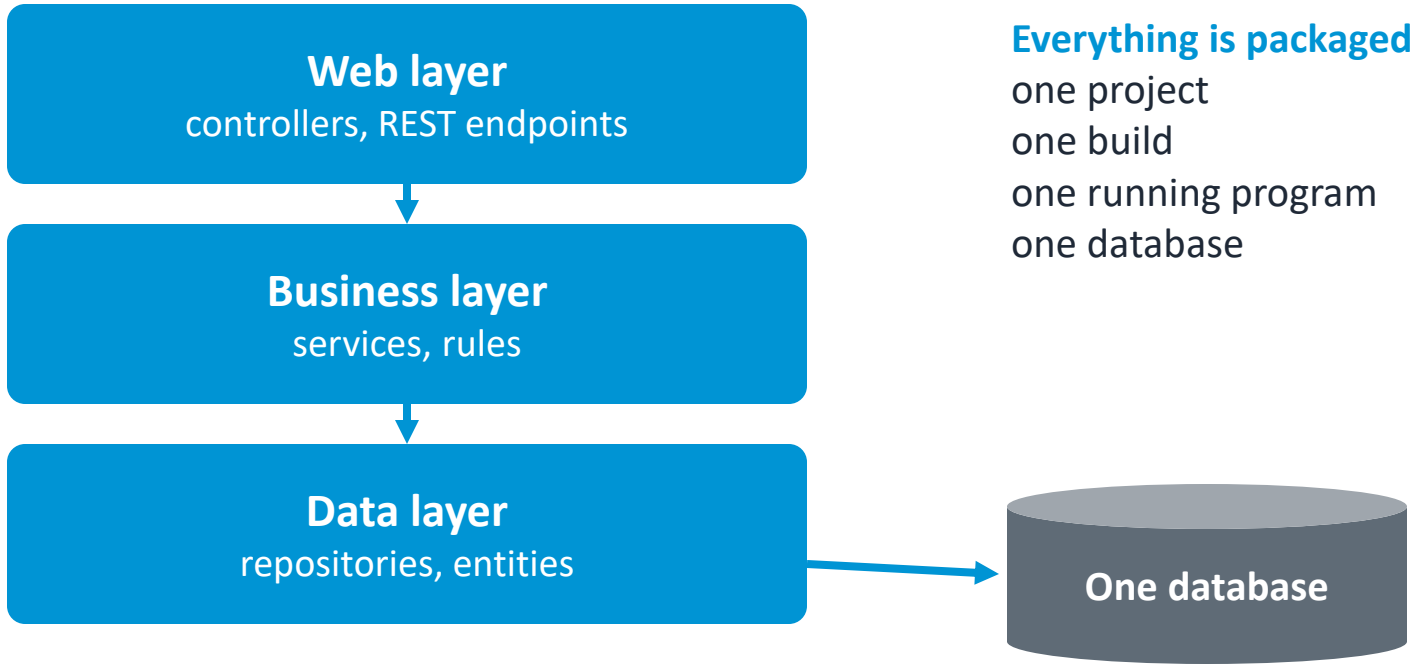
language, contexts, map

owned data, events

What can go wrong

- **Wrong cuts**
- **Shared data**
- **Chatty calls**

How an Application Is Built

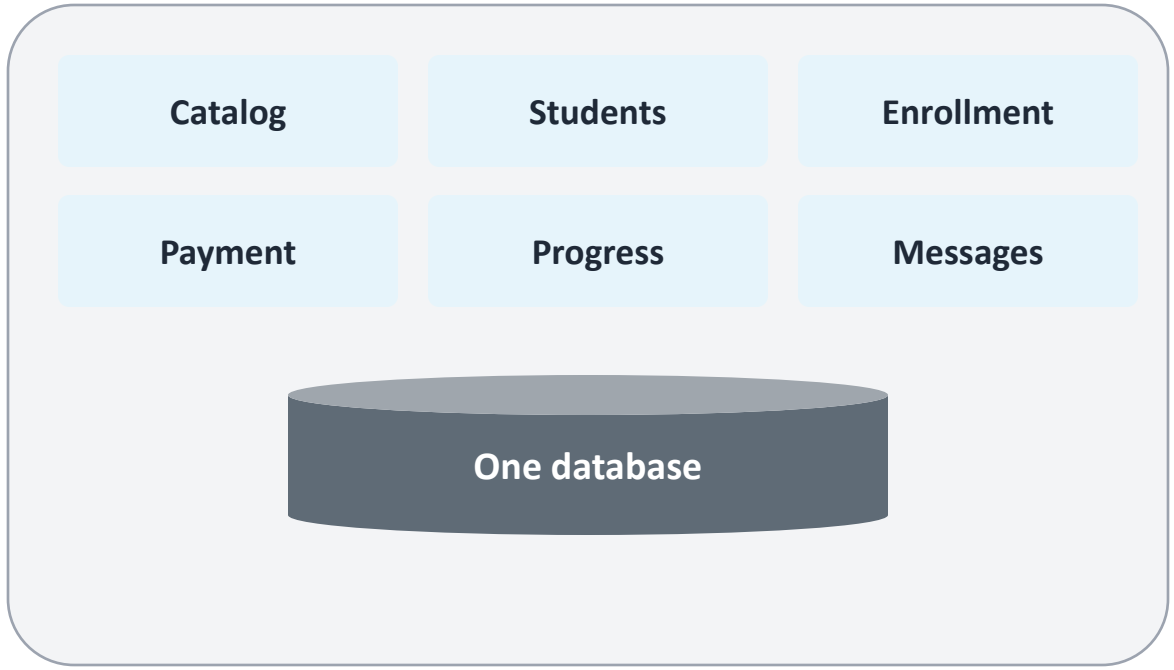


Everything is packaged and started together:
one project
one build
one running program
one database

Almost every business application has the same three layers

What Is a Monolith?

- **One codebase:** all features live in one project.
- **One deployable:** one jar or war is built, tested and released as a whole.
- **One database:** every feature reads and writes the same tables.
- **Not bad:** simple to build, run, test and debug.



One application containing every feature

One codebase, one deployable unit, one database

When a Monolith Starts to Hurt

1 Slow, risky releases

A tiny change means testing and releasing everything.

2 Tangled code

Any code can call any code; changing one thing breaks another.

3 Team conflicts

Many teams edit the same code and wait for each other.

4 All-or-nothing scaling

To handle busy lessons you must copy the whole application.

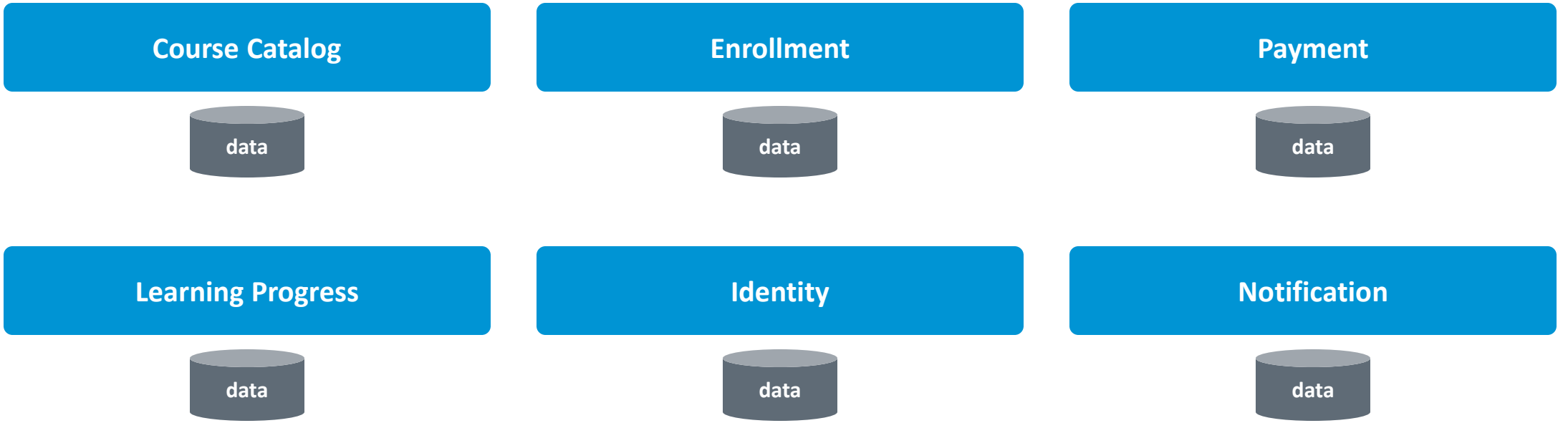
5 One failure stops all

A bug in a side feature can bring the whole site down.

The product succeeds, the team and the traffic grow, and the pain begins

What Are Microservices?

Microservices are an architecture style in which an application is built as a set of small, independent services. Each service is built around one business capability, runs on its own, owns its own data and talks to the others over the network.



Many small services, each doing one business job, each with its own data

What a Microservice Is Not

Misunderstanding	The truth
A microservice is a tiny piece of code	Small means one focused business job. It can contain many classes and operations.
Splitting code into REST controllers makes microservices	If services share one database and release together, it is still one application, only slower.
More services is better	Every service costs effort to run. Use as few as meet the need.
Microservices are always the modern, right choice	They solve certain problems and create others. A good monolith is often better.

Four common misunderstandings

Characteristics of Microservices

1. **Business-focused**

   Built around one business capability, not a technical layer.

2. **Independently deployable**

   Released on its own, without releasing the others.

3. **Owns its data**

   Its tables are private. Others ask through its API.

4. **Owned by a team**

   One team builds, runs and decides how it changes.

5. **Talks over the network**

   Through APIs or messages, never by calling its code.

6. **Fails alone**

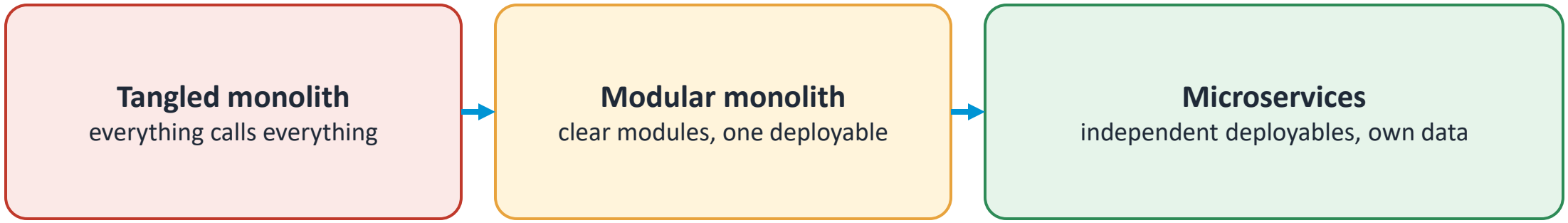
   When one service is down the others keep working, in a reduced way.

Monolith vs Microservices

	Monolith	Microservices
Deployment	One unit; every change ships everything	Each service ships on its own
Data	One database; joins and one transaction	Each service owns its data; no joins across services
Scaling	Scale the whole application	Scale only the busy service
Teams	Everyone in one codebase	One team owns one service
Failure	A bug can bring everything down	A failure can stay inside one service
Effort	Simple to build, run, test	More to run, monitor and test

Neither is better: they fit different situations

In Between: the Modular Monolith



Untangle first

Cheap to move boundaries

Extract when there is a real reason

Remember: moving a boundary inside one application is a refactoring; between services it is a project.

A well-bounded module is a service waiting to be extracted

Why Teams Choose Microservices

1. **Change independently**

   A price change in Catalog does not wait for the Payment team.

2. **Scale what is busy**

   Add copies of the lesson service on exam days only.

3. **Team autonomy**

   Teams decide their own pace, tools and release day.

4. **Fault isolation**

   If email is down, students can still enroll.

5. **Technology freedom**

   Each service may use the tool that fits; the contract is what matters.

6. **Smaller to understand**

   A new developer learns one service, not the whole system.

What Microservices Cost

Cost	What changes for you
Distributed data	No joins and no single database transaction across services; data is fetched or copied and may be briefly out of date.
The network	A method call becomes a slower call that can fail, time out or arrive twice.
Operations	Many things to build, deploy, monitor and secure instead of one.
Testing	Each service is tested alone, plus the agreements between services and a few end-to-end journeys.
Consistency	An operation that spans services becomes steps that can partly fail.
Understanding	Nobody reads the whole system in one IDE; the design documents must show how it fits.

Every service boundary is a network hop and a split of the data

Should You Use Microservices?

Consider microservices when

- several teams must release independently
- parts of the system need different scaling
- a failure in one part must not stop the rest
- the business areas are well understood
- you can run and monitor many services

Stay with a monolith when

- one small team owns the product
- the product idea is still changing every week
- you cannot yet say where the boundaries are
- you have little experience running distributed systems

Rule of thumb: start as a monolith with clear modules; extract a service when you can name the problem it solves.

Decide from the problem, not from fashion

An Analogy: One Giant Shop or a Shopping Mall

One giant shop (monolith)

- one owner and one cash desk
- cash desk breaks: everything stops
- new product: ask the owner
- simple to find and walk around

A shopping mall (microservices)

- many shops, each with its owner and till
- one shop closes: the mall stays open
- each shop decides its own stock
- needs corridors, signs and rules: the network and contracts

Lesson: independence is paid for with coordination. We must decide the shops (boundaries) carefully.

Same goods, different way to organise the business

Microservice Design Principles

1. **Single responsibility**
   one reason to change
2. **High cohesion**
   what changes together lives together
3. **Loose coupling**
   a change rarely spreads
4. **Autonomy**
   deploy, change, fail alone
5. **Own your data**
   one owner per table
6. **API is the contract**
   public surface, private inside
7. **Design for failure**
   any call can fail
8. **Team ownership**
   services follow teams
9. **Keep it simple**
   right size, start bigger

Nine questions to ask at every boundary

1. Single Responsibility

- **One reason to change:** one business capability and its rules.
- **One sentence:** say what it does without 'and'.
- **Not the same as small:** a service can have many operations.
- **Ask:** who would ask for a change here?

Student service
sign up and log in
enroll in courses
take payments
send emails
show progress

five reasons to change

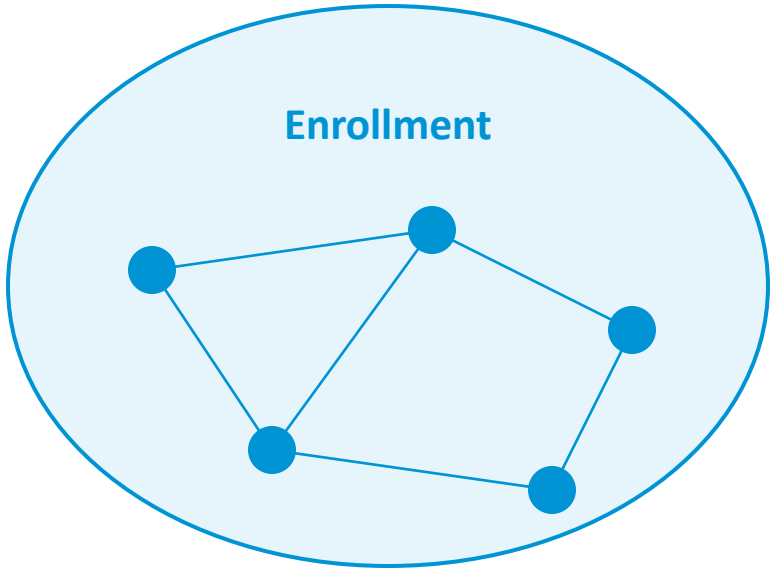
Enrollment service
Decides who may take which course: enrollments, seats and waiting list.

one reason to change

If you need 'and' to describe it, it is probably two services

2. High Cohesion

- **Cohesion:** how closely the parts inside one service belong together.
- **High cohesion:** seats, waiting list and cancellation all change when admission rules change.
- **Low cohesion:** a service with unrelated jobs that change for different reasons.

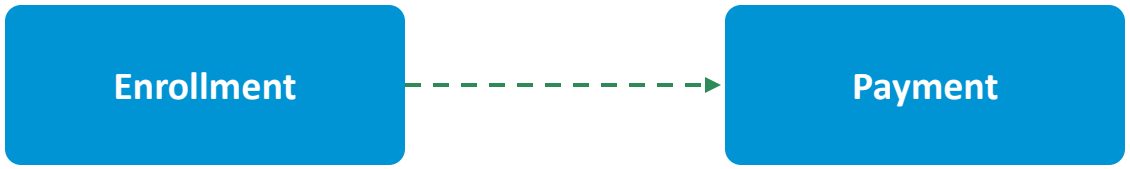


many links inside, one reason to change

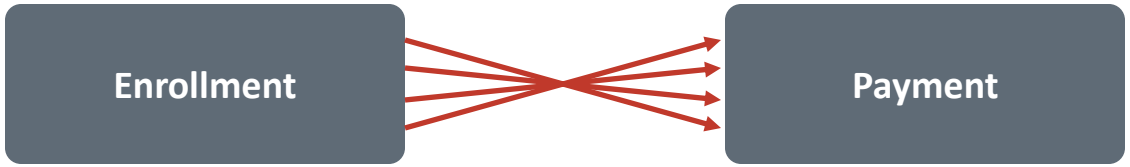
Things that change together should live together

3. Loose Coupling

- **Coupling:** how much one service must know about another.
- **Tight:** shared tables, shared code with business rules, calls that need the other side's internals.
- **Loose:** only a small, stable contract between them.



loose: one small message



tight: many calls and shared tables

A change in one service should rarely force a change in another

4. Autonomy

1

Deploy alone

A release of Payment never needs a release of Catalog.

2

Change alone

Tables, classes and libraries inside are the owner's business.

3

Fail alone

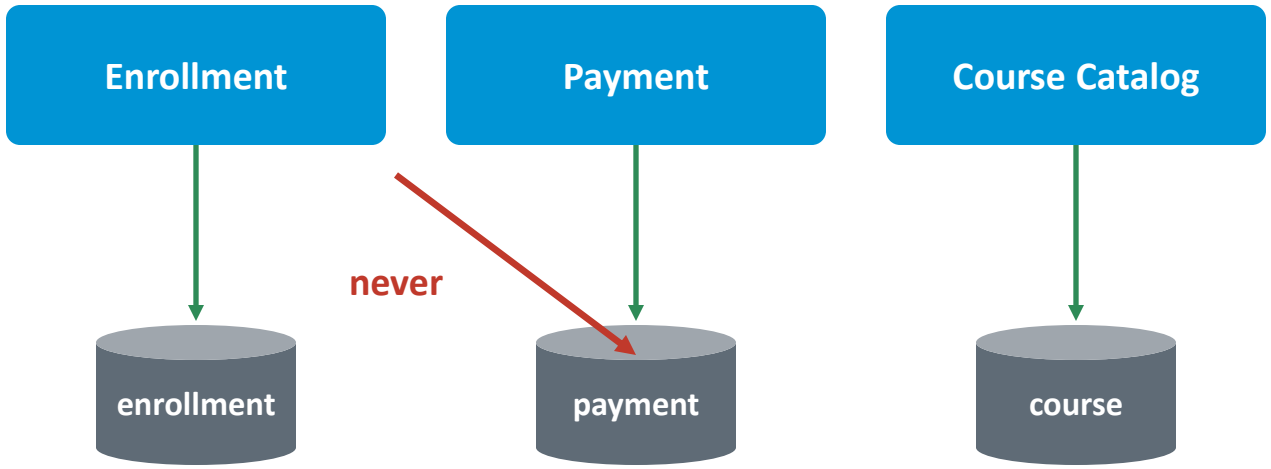
If Notification is down, students can still enroll.

Quick test: if a change in service A needs a matching change in B, or A cannot start without B, the boundary does not give autonomy yet.

If two services always ship together, they are one service

5. Own Your Data

- **One owner per table:** it decides rules and schema.
- **Database per service:** no other service reads or writes the tables.
- **Others ask:** through the API, or keep their own read copy.
- **Why:** a shared database ties all services together.

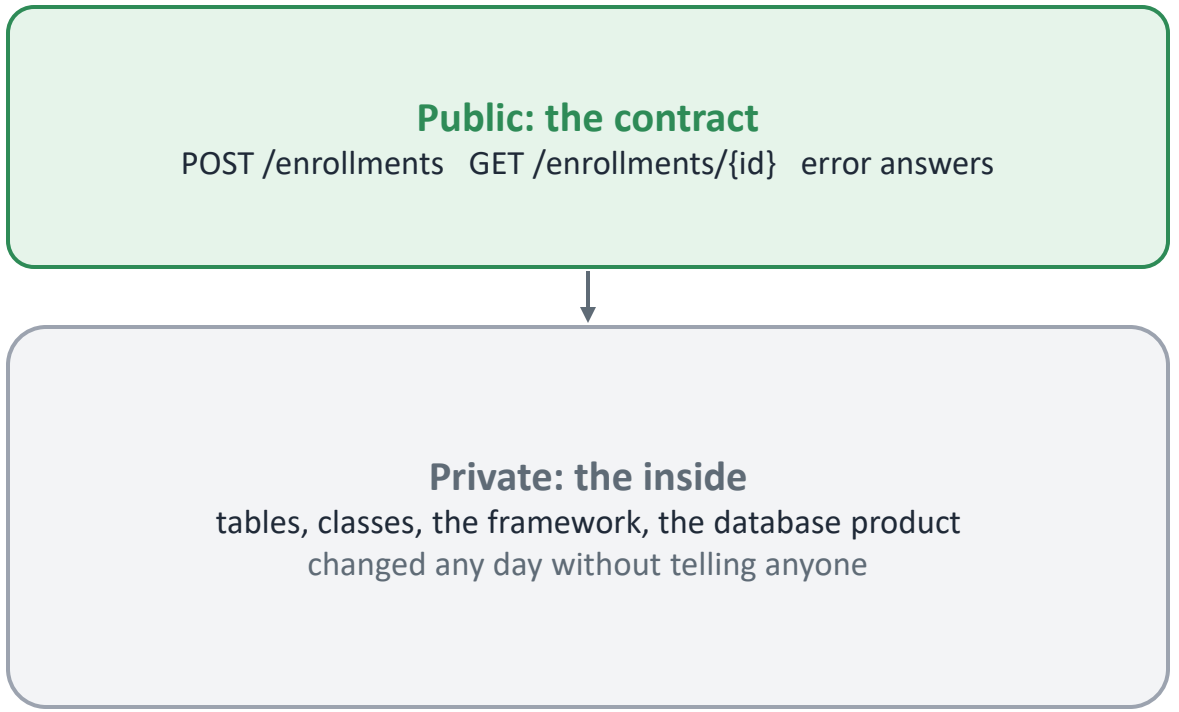


Each service has its own data store; the choice of database is private.

Only the owner touches the tables: everyone else asks

6. The API Is the Contract

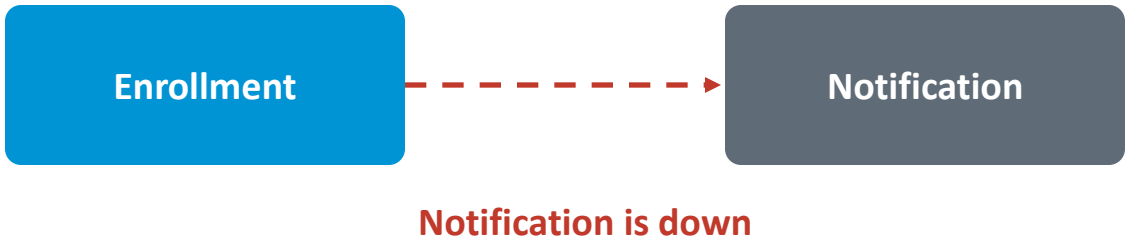
- **Public:** operations, messages, error answers and what they mean.
- **Private:** tables, classes, libraries, language, framework.
- **Design it first:** decide what others need before building.
- **Keep it small:** every public item is a promise.



Others depend on the contract, never on the inside

7. Design for Failure

- **Expect it:** networks fail and services restart.
- **Degrade, do not stop:** if Notification is down, enrollment still works.
- **Prefer messages:** when the other side can act a little later.

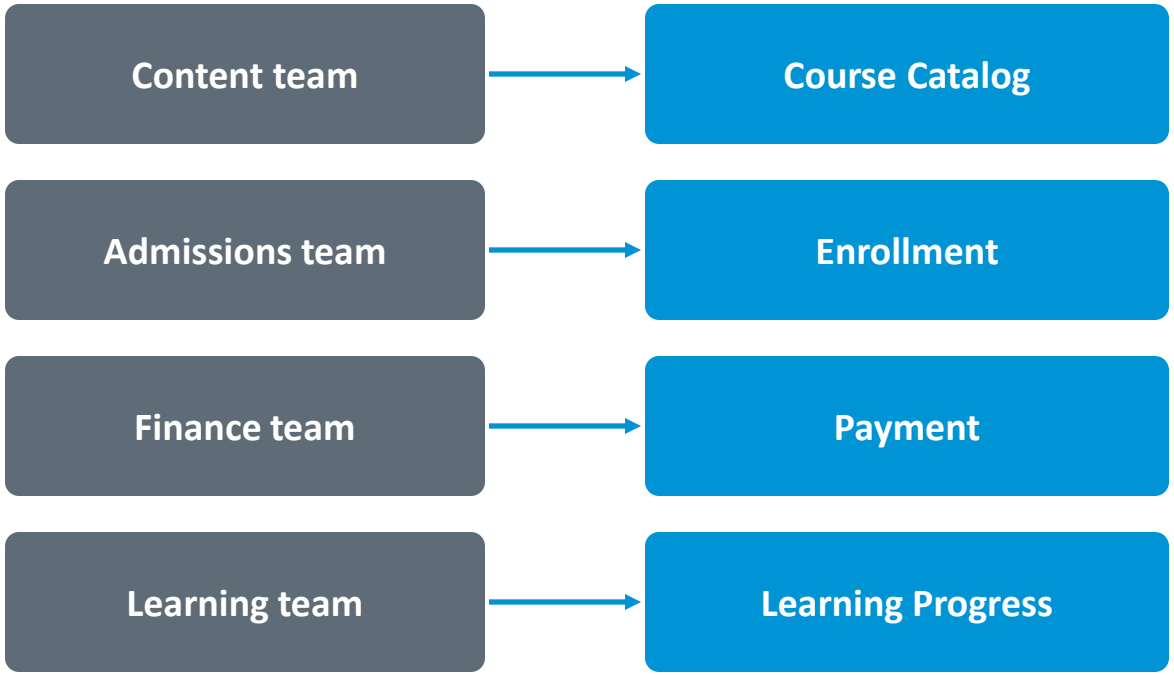


The student is still enrolled.
The message is sent later, when the service is back.

A call to another service can be slow, fail or arrive twice

8. Team Ownership

- **You build it, you run it:** the team owns design, code and operation.
- **One team, one or a few services:** never one service shared by many teams.
- **Boundaries follow the organisation:** a service nobody owns decays.
- **Test:** who is called at night when it breaks?



Services follow teams: one service, one owning team

9. Keep It Simple: the Right Size

Too big

- many reasons to change
- several teams inside
- releases wait for each other

Right size

- one sentence, one team
- owns its data
- works, reduced, when others fail

Too small

- one feature needs five calls
- services cannot work alone
- more services than people

Rule of thumb: if you cannot say why a service must be separate, keep it inside another service for now.

Too big is a monolith again; too small is a mess of calls

Three Boundary Tests

1 One sentence

Describe the service in one sentence without 'and'.

Enrollment decides who may take which course.

2 One reason to change

One team asks for changes and the changes do not need another service.

Finance changes refund rules: only Payment is touched.

3 Own data and failure

It owns its tables, and still works, reduced, when others are down.

Notification is down: enrollment still works.

A good boundary passes all three

The Big Question: Where Do We Cut?

- **Wrong cuts:** break the principles and make every change cross services.
- **Code alone does not tell us:** the tables and classes show how it is built, not how the business works.
- **We need:** a shared understanding of the business, and a way to draw lines in it.

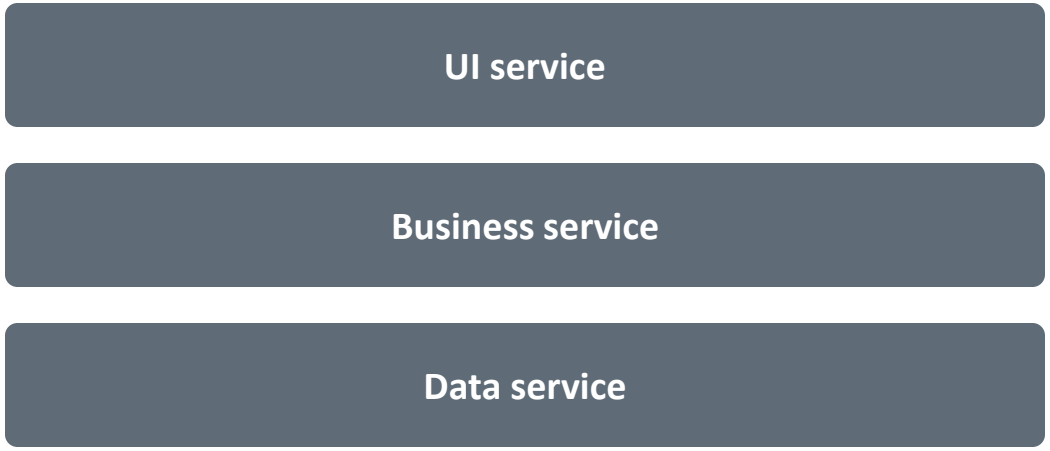


Where does Enrollment end and Payment begin? What is 'a course'?

Everything so far assumed the boundaries are right. How do we find them?

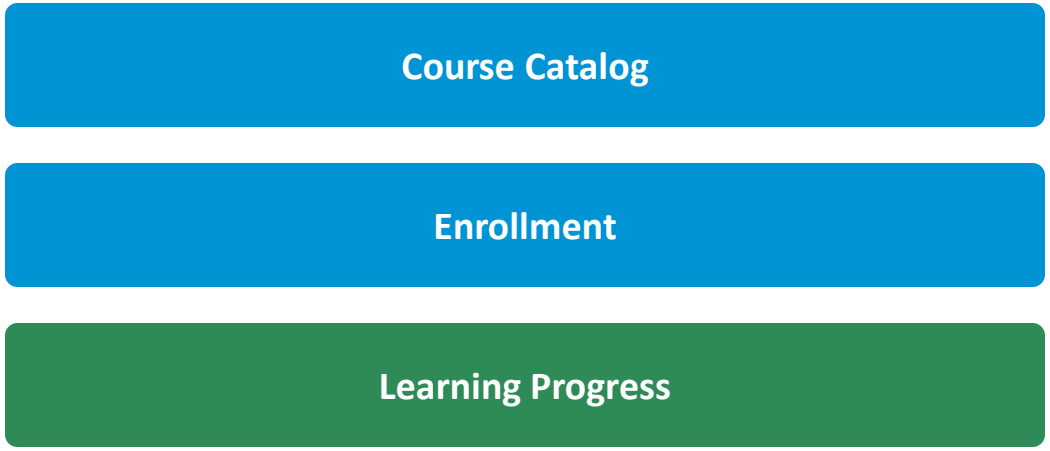
Cutting by Layer or by Business?

Cut by layer (wrong)



Change: a certificate needs 80 percent progress
touches all 3 services

Cut by business (right)

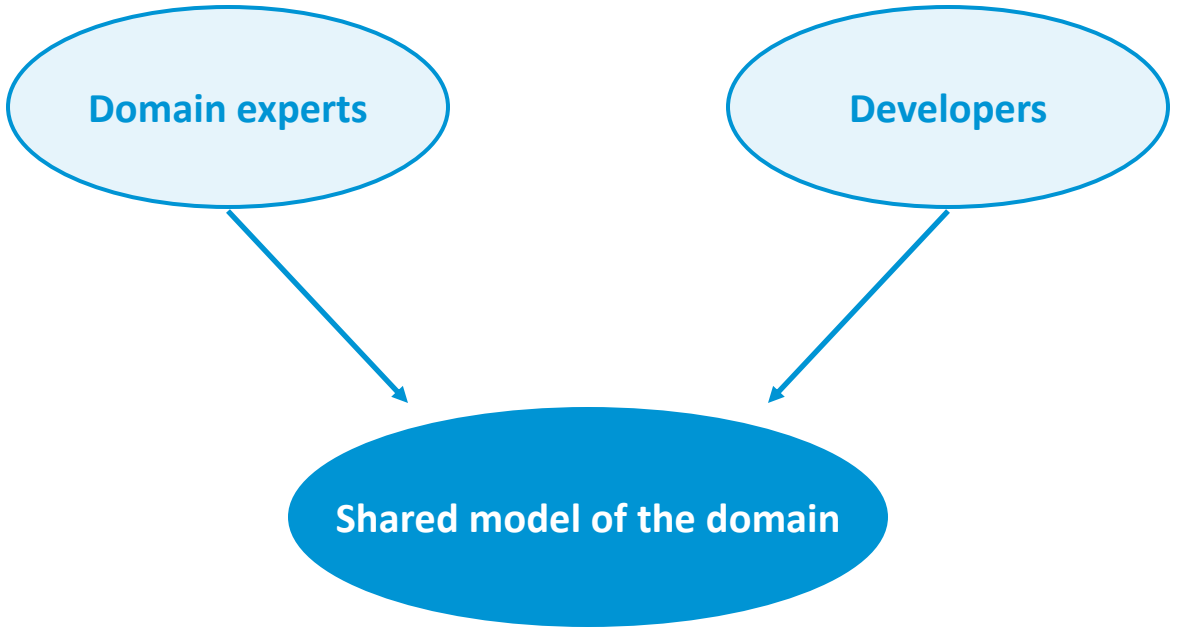


Same change
touches 1 service: Learning Progress

Cut along the business, not along the technology

What Is Domain-Driven Design?

- **Domain:** the business area the software serves, for example online learning.
- **Domain experts:** people who know the business.
- **DDD:** developers and domain experts model the business together, in a shared language.
- **Result:** software that mirrors the business, and natural boundaries.



Understand the business first, then build the software around it

DDD Has Two Parts

Strategic design: the problem space

What is the business? Why?

- ubiquitous language
- domain and subdomains
- core, supporting, generic
- bounded contexts
- context mapping

Tactical design: the solution space

How do we build it?

- entities and value objects
- aggregates
- domain events
- application services

Strategic design finds the boundaries; tactical design shapes what is inside

Ubiquitous Language

- **One word, one meaning:** write a glossary.
- **Use it everywhere:** meetings, code, tables, APIs.
- **It is built together:** domain experts and developers.

Term	Meaning
Student	A person who learns on the platform
Course	A learning programme offered on the platform
Enrollment	A student's right to take a course
Seat	One place in a course batch
Payment	Money received for an enrollment
Progress	The lessons a student has completed

The same word means the same thing for developers and business people

Domain and Subdomains

Domain: online learning

Course catalog
subdomain

Enrollment
subdomain

Payment
subdomain

Learning progress
subdomain

Identity and access
subdomain

Notification
subdomain

A business area is made of smaller areas

Core, Supporting and Generic Subdomains

Core

Why customers choose you. Build it well, in-house.

Learnly: course catalog, learning progress

Supporting

Needed, but not what makes you different. Keep it simple.

Learnly: enrollment, payment

Generic

Every business has it. Use a ready-made tool if you can.

Learnly: identity, notification, audit log

Not every part of the business deserves the same effort

Demo: Write the Language and Sort the Subdomains

What we do

- **Write:** eight words of the Learnly glossary.
- **Sort:** subdomains into core, supporting, generic.
- **Say:** what we build, keep simple or buy.

What it shows

- **One language:** no word with two meanings left unnoticed.
- **Priorities:** effort goes to the core.
- **Input for contexts:** next demo.

Bounded Context: One Word, One Meaning

In Course Catalog

A course is a title, a syllabus, an instructor and a price.

In Enrollment

A course is something with a number of seats and a waiting list.

In Payment

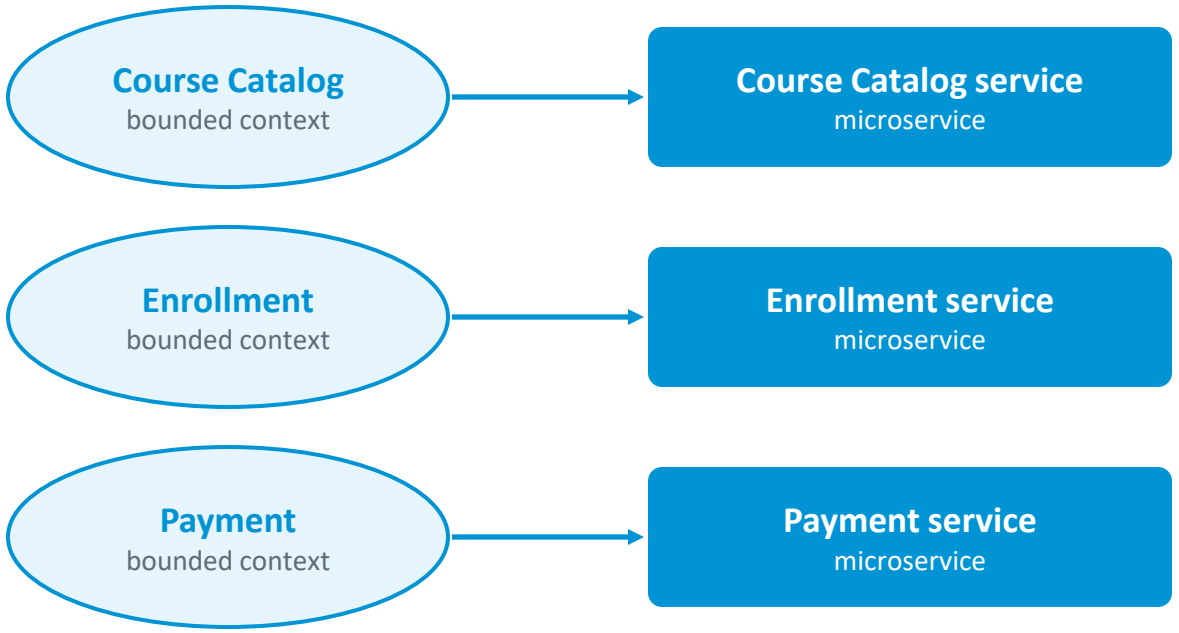
A course is a line on an invoice with the amount paid.

Each context keeps its own model of Course. They share only the course id.

The same word can mean different things in different parts of the business

From Bounded Context to Microservice

- **Same shape:** own language, own rules, own data, one team.
- **So:** start with one bounded context per microservice.
- **Not a law:** two small contexts may share a service, a large one may split.
- **The context is the design; the service is its deployment.**



A bounded context is a natural candidate for one service

Describing Bounded Contexts

Bounded context	Responsibility	Owns data	Works with
Course Catalog	Tells students which courses exist and what they cost.	Course, SyllabusItem	Enrollment, Notification
Enrollment	Decides who may take which course.	Enrollment, SeatBatch	Course Catalog, Learning Progress, Notification, Payment
Payment	Takes money for an enrollment and gives it back.	Payment, Refund	Enrollment, Notification
Learning Progress	Follows a student through a course and certifies the result.	LessonProgress, Certificate	Enrollment, Notification
Identity & Access	Knows who a student is and lets them sign in.	Student, Credential	Notification
Notification	Sends messages to students.	Template, MessageLog	Course Catalog, Enrollment, Identity & Access, Learning Progress, Payment

A table says what each context is responsible for and whom it works with

Demo: Draw the Bounded Contexts

What we do

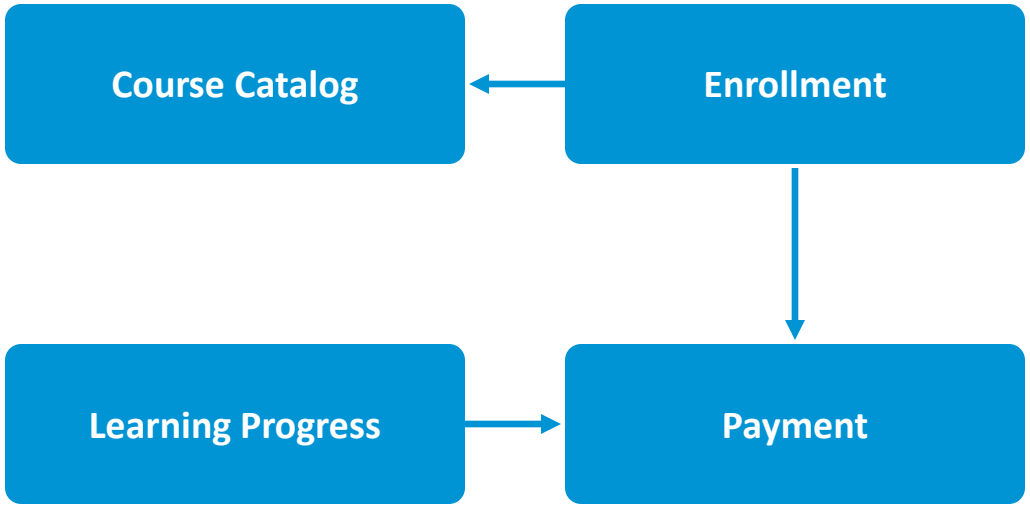
- **Group:** features that share a language and change together.
- **Name:** each group, with a one-line responsibility.
- **Test:** the three boundary tests.

What it shows

- **Contexts from features:** not from tables.
- **The table:** responsibility, data and neighbours.
- **Moves:** what fails a test is moved or merged.

Context Mapping

- **Context map:** a drawing of the contexts and how they depend on each other.
- **Each arrow:** one context needs something from another.
- **For services:** every arrow becomes an API call or a message.
- **Why draw it:** shows loops and long chains at a glance.



arrows: who needs something from whom

A context map shows how the contexts relate to each other

Ask or Be Told

Ask: call and wait

Enrollment asks Course Catalog: is the course open, and what does it cost?
Use when you need the answer right now.
Cost: if Course Catalog is down, Enrollment waits or fails.

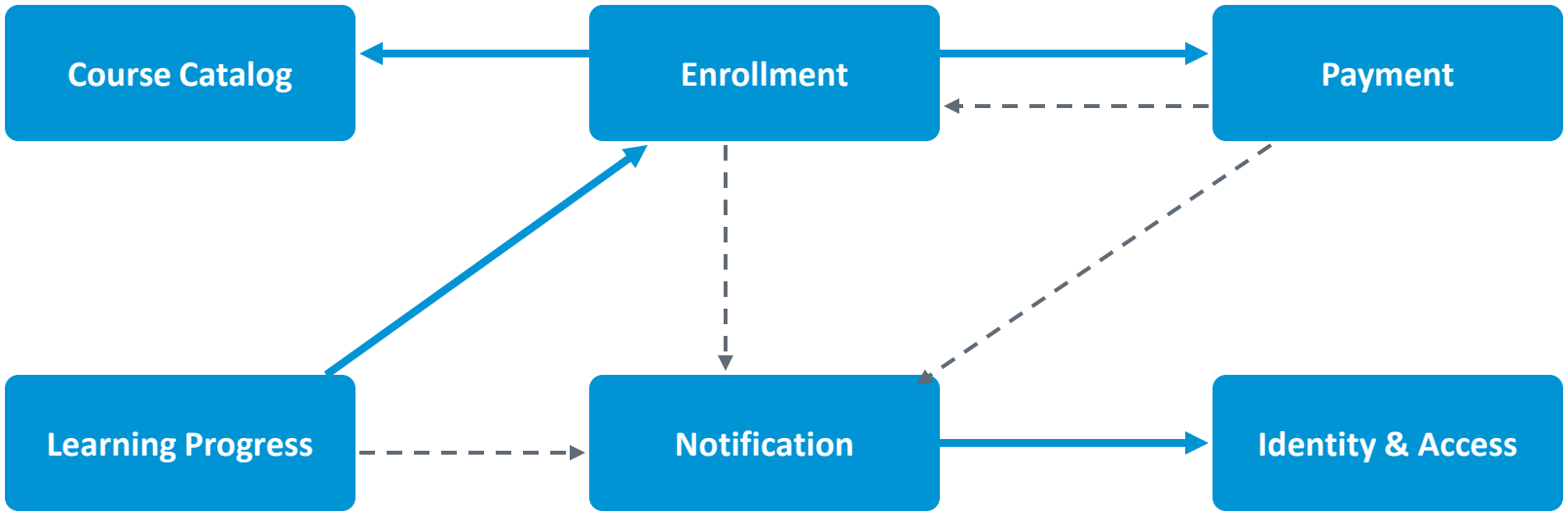
Be told: receive a message

Payment tells Enrollment: the payment completed.
Use when the other side can act a little later.
Cost: the news may arrive late.

Two rules: no loops (A asks B, B asks A) and keep chains of asking short, at most two.

Two ways for services to depend on each other

The Context Map of Learnly



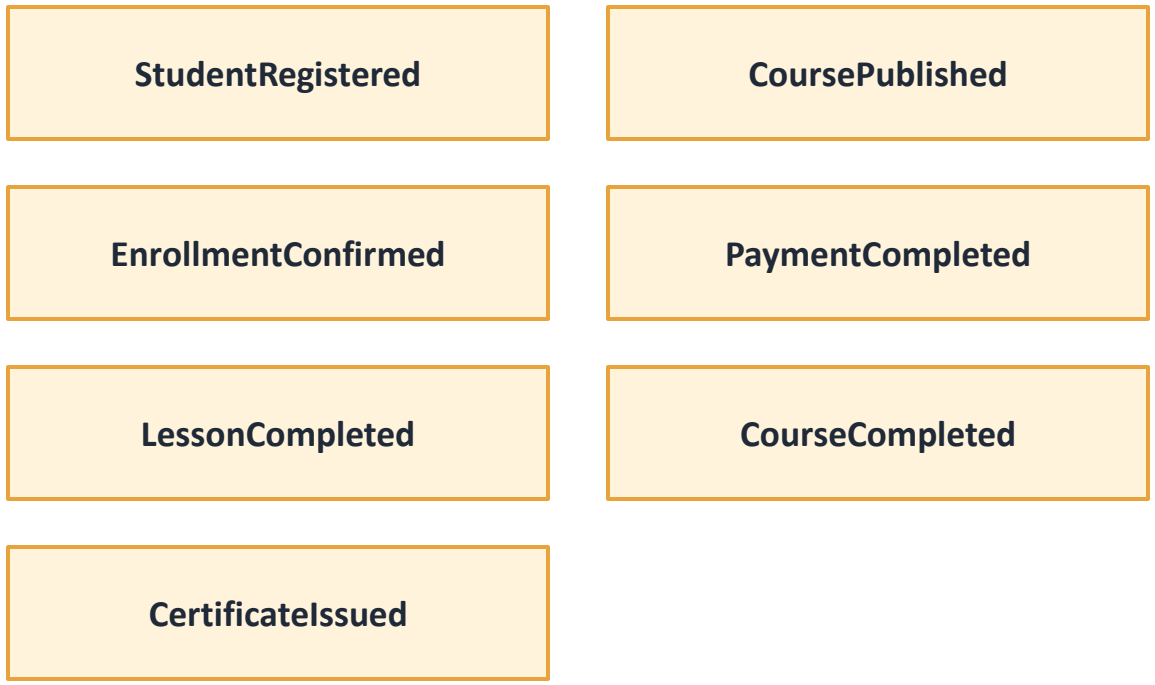
Legend
solid: asks and waits
dashed: told by message
4 asks, no loops, longest chain 2

Why draw it: loops and long chains are visible at a glance, and new people read it first.

Solid arrow: asks and waits. Dashed arrow: is told by message

Collaboration and Event Storming

- **Domain experts and developers:** in the same room.
- **Event storming:** write events in past tense, in order, on sticky notes.
- **Look for clusters:** groups of events with the same words are contexts.
- **Keep talking:** the model changes when the business changes.



Boundaries are found with business people, not for them

Demo: Draw the Context Map

What we do

- **Draw:** a box for each bounded context.
- **Connect:** solid arrows for asks, dashed for messages.
- **Check:** loops, chain length and shared data.

What it shows

- **One picture:** all the context together.
- **Loops and chains:** visible on the drawing.
- **Ready:** arrows become calls and messages.

Inside a Service: Entities and Value Objects

- **Entity:** has an identity that lasts. Student s-77, Course c-305.
- **Value object:** no identity, only values. Money, Address, EmailAddress.
- **Why it matters:** entities are the data a service owns.

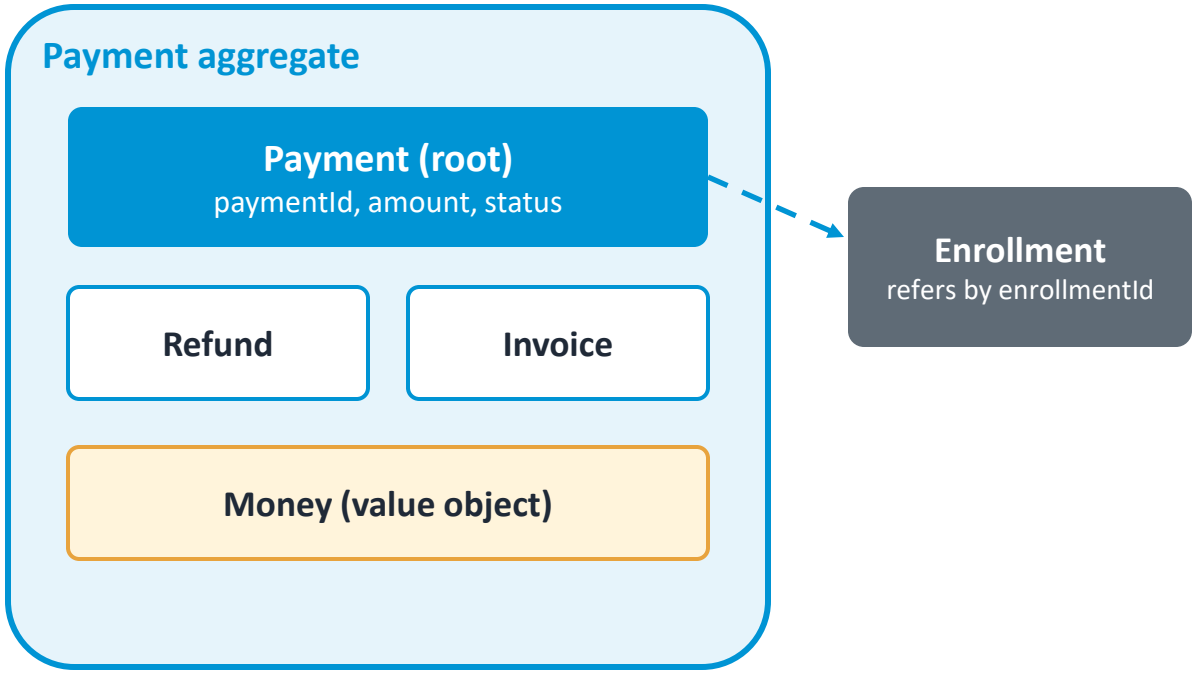
Entity: Course
courseId c-305
title
price
status
same id, same course, even if the price changes

Value object: Money
amount 500
currency INR
no id: two equal values are the same thing

What the service keeps, and how we tell the pieces apart

Aggregates

- **Aggregate:** objects that must stay consistent together.
- **Aggregate root:** the only entry point from outside.
- **One transaction:** one aggregate.
- **Other aggregates:** referred to by id only.



A group of objects changed together, with one entry point: the aggregate root

Domain Events and Repositories

- **Domain event:** something that happened, in past tense.
- **Becomes a message:** the service tells; others may react.
- **Repository:** stores and loads one kind of aggregate.

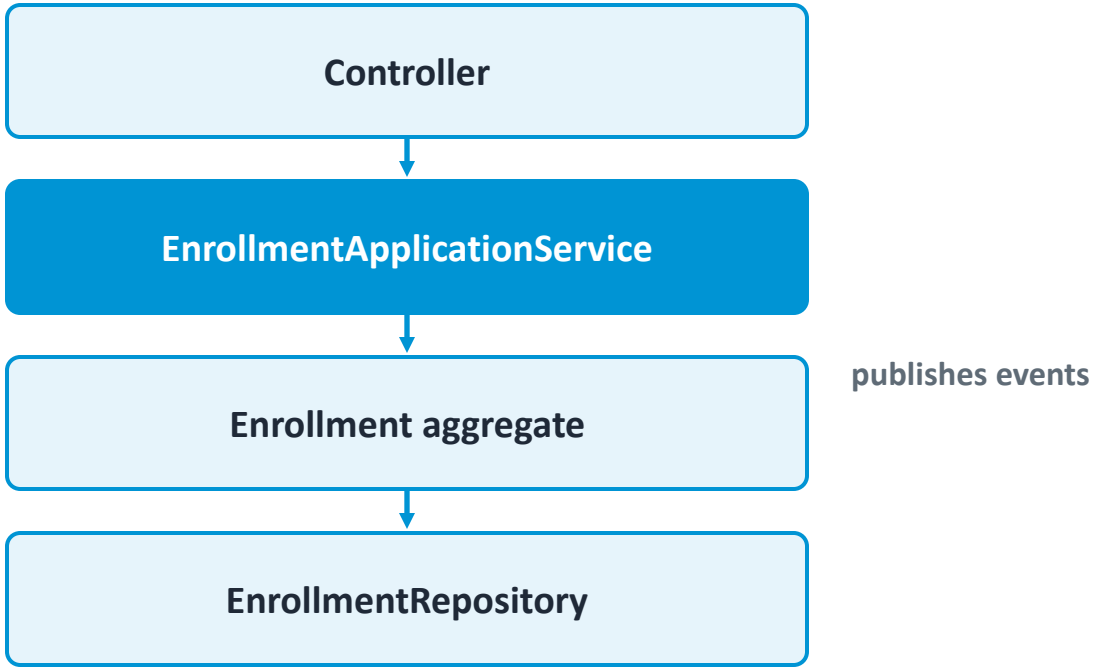


Event	Meaning
EnrollmentConfirmed	A seat is taken and paid
CourseCompleted	A student finished all lessons

Events are what a service tells others; a repository is how it keeps its entities

Application Services: the Use Cases

- **Use case:** one thing a user or another service asks for.
- **Application service:** loads, asks the aggregate, saves, publishes events.
- **Rules live in the domain:** the service only coordinates.
- **In Spring:** the class between controller and repository.



One place per service where a business request is carried out

The Service Worksheet

Part	What to write
Purpose	One sentence, without 'and'
Responsibilities	Two to four capabilities
Does not do	What people may expect but belongs elsewhere
Entities and value objects	The data it owns; ids it keeps of others
Aggregate and root	What changes together
Asks	Whom it calls and waits for
Events it publishes	What it tells, past tense
Application services	Use cases it carries out
Team	Who owns it

One worksheet per service: everything it is responsible for

Example: the Enrollment Worksheet

Part	Enrollment
Purpose	Decides who may take which course.
Responsibilities	Enroll a student; Enforce seat limits and the waiting list; Cancel an enrollment; List a student's courses
Does not do	course prices, money, lesson progress
Entities	Enrollment (studentId -> Identity & Access.Student, courseId -> Course Catalog.Course); SeatBatch (courseId -> Course Catalog.Course)
Aggregate (root)	Enrollment with EnrollmentStatus
Asks	Course Catalog: is the course open, and what does it cost?; Payment: start the payment for an enrollment
Publishes	EnrollmentConfirmed; EnrollmentCancelled
Application services	EnrollmentApplicationService: enroll, cancel, join the waiting list, list courses of a student
Team	Admissions team

Demo: Describe What Is Inside Each Service

What we do

- **Entities:** the data of each context, with ids of others.
- **Aggregate and events:** what changes together, what it tells.
- **Use cases:** one application service per context.

What it shows

- **A worksheet per service:** responsibilities made concrete.
- **Ids not joins:** references written down.

The Method in Six Steps

1 Understand

Talk to experts; list features and pains.

2 Language

Write the glossary.

3 Subdomains

Core, supporting, generic.

4 Contexts

Group, name, one-line responsibility.

5 Map

Arrows: ask or be told; no loops.

6 Describe

Worksheet per service.

From a business story to a set of services

Review Checklist for a Service Design

- ✓ A glossary exists and the code uses it
- ✓ Every table has exactly one owner, others keep ids
- ✓ Every subdomain is core, supporting or generic
- ✓ No loop between services; chains of asks at most two
- ✓ Every service has a one-sentence purpose without 'and'
- ✓ Every event has one producer and named consumers

Key Takeaways

1. **Microservices**
   Small independent services, own data, one business job each.

2. **Principles**
   One responsibility, cohesion, loose coupling, autonomy, own data.

3. **Where to cut**
   Along the business, using DDD, not along layers.

4. **Language and contexts**
   Glossary, subdomains, one bounded context per service.

5. **Map and inside**
   Ask or be told, no loops; entities, events, use cases.



Happy Learning :)